

Coursework 1

COS6012-B - Concurrent and Distributed Systems

UB Number: 25029204

Question 1 - interference

Explain the concept of interference in concurrent systems and provide an example of how it could occur within the context of this scenario.

In concurrent systems, interference occurs when multiple threads access shared resources without proper synchronization, causing unexpected or incorrect results (FRIESEN, 2015). In other words, the outcome can suffer from interference when different processes try to read and write the same resource, manipulating it wrongly and overwriting data. For example, in Betty's Café, if two customers try to order the last slice of cake at the same time, without proper synchronization, both customers might end up being served the same slice, leading to a conflict and an incorrect output of the system. Technically speaking, the shared resource (the last slice of cake) is accessed by multiple threads (the customers) without proper coordination, resulting in an inconsistent state.

Question 2 - deadlock

Describe what a deadlock is, explaining its causes and the strategies used to avoid it.

A Deadlock is a state in concurrent systems where two or more threads are blocked forever, waiting for each other to release resources to continue their execution (Javier Fernandez Gonzalez, 2017). For example, in Betty's Café, if the staff stop to prepare new items and one customer is waiting for a cup of tea while holding the last slice of cake while another customer is waiting for a slice of cake while holding the last cup of tea, both customers will be stuck in a deadlock, as they are waiting for each other to release the resources they need.

For a deadlock to occur, four conditions must be present simultaneously. The first is the "Mutual Exclusion" concept, where only one thread can access a resource at a time. Secondly, the "Hold and Wait" condition explains how a thread holds one resource and waits for another, still not releasing the one that it already has. The third condition "No Preemption" defines that resources cannot be forcibly taken from threads, and the final one is called "Circular Wait": a chain of threads exists where each thread is waiting for a resource held by the next thread in the chain (M Raynal, 2013).

To avoid deadlocks, it is necessary to break at least one of these conditions. One common approach is to use a lock hierarchy (also known as lock ordering), where all threads acquire locks in a predefined order. This prevents circular wait conditions, as threads will always acquire locks in the same sequence. For example, if the café staff always acquire locks for the tea before acquiring locks for the cake, and customers do the same, it will prevent incorrect manipulation of the items. This is especially important in situations where the order of locks depends on method arguments, such as `getOrder(cake, tea)` and `getOrder(tea, cake)`, which can lead to deadlocks if the locking order changes dynamically.

Another strategy is to implement a timeout mechanism, where threads give up waiting for a resource after a certain period. This solution does not guarantee that a contention will never happen, but it allows the system to recover from potential deadlocks. Additionally, using non-blocking algorithms and thread safe structures such as `ConcurrentHashMap` or `BlockingQueue` can also help prevent deadlocks by avoiding the use of locks altogether.

To conclude, it is important to say that even if a deadlock happens, it is possible to detect and recover from it. The system can periodically check for deadlocks by analyzing the state of threads and resources. If a deadlock is detected, the system can take corrective actions, such as terminating one of the threads involved (to release resources) in the deadlock or rolling back some operations to break the circular wait.

Question 3 - deadlock in this scenario

Explain how a faulty implementation of this scenario could lead to a deadlock, providing a concrete example. Then propose a suitable strategy to prevent it.

A faulty implementation of the threads responsible for staff and customers could lead to a deadlock if they do not follow a proper locking order when accessing shared resources. This happens because deadlock occurs when threads are stuck waiting for each other to release these resources, causing a circular wait (Wittman, Mathur and Korb, 2013). In particular, an example that shows how deadlock arises is a context where clients try to order teas and cakes (decreasing the number of available items), while staffs try to access the same components to prepare them (increasing the amount of available elements).

In this scenario, a circular wait is likely to occur if staff A acquires a lock for tea and then waits for a lock for cake, while customer B acquires a lock for cake and then waits for a lock for tea, both threads will be stuck in a deadlock. Consequently, staff A will be waiting for customer B to release the lock for cake, while customer B will be waiting for staff A to release the lock for tea. This circular wait condition leads to a deadlock, as neither thread can proceed until the other releases the resource it holds.

To prevent this, a suitable strategy would be to establish a consistent locking order for all threads. For example, the café could enforce a rule that all threads must acquire locks in the order of tea first, then cake. This way, if staff A acquires the lock for tea, it will not wait for the lock for cake while customer B is holding it, as customer B will also be required to acquire the lock for tea first before acquiring the lock for cake. This removes the circular wait condition, breaking one of the necessary conditions for deadlock.

Question 4 - System design and concurrency guarantees

Describe how you would design your solution for this scenario. Identify the threads and the main classes that you would define, and explain which concurrency concepts and Java synchronisation mechanisms you would use, and why. Your design must: Ensure mutual exclusion when accessing shared resources; Be free from deadlock and livelock; Incorporate fairness; Provide detailed justification for how your design satisfies each of these requirements.

First of all, the implementation of the solution for Betty's Café involves defining a program that simulates a spot where people come to enjoy food, drinks and music while the staff keep the café running by preparing the buffet items. In this context, there are two main types of threads: customers and staff. The

customers will perform random actions such as ordering, eating, drinking, playing the piano, and listening to music, while the staff will be responsible for preparing coffee and cake for the buffet.

Second of all, there is a `Main` class that will serve as the entry point of the program, where the café session is initiated (to tell everyone when the café is open or closed) as well as a shared resource class `Buffet` that holds counts of cakes, teas, and coffees, and a `Piano` class that only lets two people play at once is instantiated. Then it starts a few threads for customers and staff and let them run for fixed amount of time. When the session time is up, it marks the session closed, tells the buffet to wake everyone up, and politely interrupts all the threads so the program shuts down cleanly.

Furthermore, it is important to highlight how the shared resources are handled. The class `Buffet` represents the shared resources of the café, such as the numbers of cakes, teas, and coffees. To make sure these numbers don't get messed up when several threads touch them at once, every method that reads or changes them grabs a lock first. While using a `ReentrantLock` created with the "fair" option (which make threads take turns in the order they asked rather than barging in), the access to the shared resource guarantees mutual exclusion by ensuring that only one thread access the inventory at a time.

Additionally, when an item is unavailable, the customer thread will wait until the staff thread prepares the item and updates the inventory, using a `Condition stockChanged` to signal waiting threads when an item becomes available, avoiding the need for busy waiting. Whenever staff adds stock, they call `signalAll()` to wake every waiting customer so they can try again. Besides that, the availability check and all inventory modifications are performed within the same critical section to ensure atomicity and prevent race conditions.

Moreover, if a customer wants to play the piano, the `Piano` class allows them to do so in a safe way. The class has a method using a `Semaphore` with fairness turned on and limits the number of customers who can play at the same time to 2. The count of how many people are currently playing is shown through an `AtomicInteger` that is updated atomically. This ensures that the piano is not accessed by more than 2 customers simultaneously, preventing interference and ensuring the correct access control.

Focusing on the threads, the `Customer` thread class that represents the actions of a customer, such as ordering, eating, drinking, playing the piano, and listening to music. There is also a `Staff` thread class that represents the actions of a staff member, such as adding random new items to the buffet.

To control the session, there is a class `CafeSession`, which manages the lifecycle of the café session, including starting and ending the session, and ensuring that all threads are properly managed during this time. This class would manage a flag as a `AtomicBoolean open` used by the staff's threads to determine if the session is still active. When the session ends, the `CafeSession` class will set this flag to false, signal all waiting threads that the buffet is closed with `buffet.signalClosed()` allowing them to exit. Then, all customer and staff threads are interrupted to ensure a clean shutdown. This design ensures that all threads can terminate gracefully when the café session ends, preventing any threads from being left hanging indefinitely.

As for preventing deadlock, this design never holds more than one shared resource at a time. This means that customers acquire the buffet lock to only check and atomically take elements from the inventory, then release the lock before proceeding to other operations, such as playing the piano. As no code ever tries to hold more than that one lock at a time, there's no chance of a circular wait between threads.

Customers release the lock when they wait, so staff can always get it to add items. Therefore, there is no hold-and-wait across multiple resources, eliminating circular wait and making deadlock impossible.

Livelock occurs when threads remain active and repeatedly retry operations without making progress (for example, through repeated `tryLock` attempts). In this design, customers do not actively retry, instead, they block using `stockChanged.await()` until the entire order becomes available and then proceed when the `signalAll()` method is called. Because waiting is blocking rather than repeatedly retrying, threads do not interfere with each other and livelock is avoided as at least one waiting thread will make progress.

As for fairness, both the buffet lock and the piano semaphore were created with `true` passed for the fairness parameter. This ensures that threads acquire locks in the order they requested them, in a First-In-First-Out (FIFO) manner. This prevents starvation by serving waiting threads in arrival order. Combining this with the `signalAll()` method, all threads are given a fair chance to access the shared resources.

In conclusion, overall, this design ensures safety (mutual exclusion) and liveness (progress without deadlock, livelock, or starvation). The program keeps shared data safe by locking, avoids deadlock by sticking to a single lock, wakes threads properly so they don't get stuck forever, and treats waiting threads fairly so everyone gets a chance. The result is a simple but correct little simulation where customers and staff peacefully cooperate.

Example of simulation output

```
Starting program with 6 clients and 3 staff
Buffet = (2 cakes, 2 teas, 2 coffees).
Client-1 listens to music for 722ms.
Client-4 wants tea.
Client-2 wants cake.
Client-5 wants coffee.
Client-3 wants tea.
Client-4 takes tea.
Client-6 wants tea and cake.
Buffet = (2 cakes, 1 tea, 2 coffees).
Client-2 takes cake.
Buffet = (1 cake, 1 tea, 2 coffees).
Client-4 drinks for 2698ms.
Client-3 takes tea.
Buffet = (1 cake, 0 teas, 2 coffees).
Client-3 drinks for 2349ms.
Client-2 eats for 1545ms.
Client-5 takes coffee.
Buffet = (1 cake, 0 teas, 1 coffee).
Client-5 drinks for 1989ms.
Client-6 waits.
Client-1 finished listening music.
Client-1 wants coffee.
Client-1 takes coffee.
Buffet = (1 cake, 0 teas, 0 coffees).
Client-1 drinks for 931ms.
Staff-3 brings 3 coffees.
Buffet = (1 cake, 0 teas, 3 coffees).
Staff-3 returns to kitchen.
Staff-1 brings 3 teas.
Buffet = (1 cake, 3 teas, 3 coffees).
Staff-1 returns to kitchen.
Client-6 takes tea and cake.
Buffet = (0 cakes, 2 teas, 3 coffees).
```

```
Client-3 wants to play the piano.
Client-3 plays the piano for 1463ms. (active players=1)
```

With this output, it is possible to see that the café session starts with 6 clients and 3 staff members. The buffet is initiated with 2 cakes, 2 teas, and 2 coffees.

Customers are listening to music, ordering items, some are playing the piano, and staff are adding items to the buffet.

Looking carefully, it is possible to see that the 4th line shows that the first customer to order an item is Customer 4, who wants tea. Others customers make their orders as well, but the first customer to succeed is Customer 4, as shown in line 8. Then, the buffet is updated and logged in line 10, that shows the buffet with one less tea.

The cafe behaviour continues. A important point to notice is that when it is the turn of Customer 6 to take a tea and a cake, the buffet has none tea left, so the customer waits until staff member 1 (line 21) adds more teas to the buffet, as shown in line 30. Then, the customer can finally take the tea and cake, as shown in line 33.

```
Session is closing.  
Client-3 leaves the cafe.  
Client-1 leaves the cafe.  
Client-2 leaves the cafe.  
Client-4 leaves the cafe.  
Staff-3 ended shift.  
Staff-1 ended shift.  
Staff-2 ended shift.  
Client-5 leaves the cafe.  
Client-6 leaves the cafe.  
Session ended. Active threads=0.
```

Finally, the café session ends, as shown in the final output. The buffet is closed and all threads are interrupted, so the program shuts down cleanly.

References

FRIESEN, J. (2015). Java Threads and the Concurrency Utilities. Apress.

Javier Fernandez Gonzalez (2017). Java 9 Concurrency Cookbook -. Packt Publishing Limited.

M Raynal (2013). Concurrent programming : algorithms, principles, and foundations. Heidelberg ; New York: Springer-Verlag.

Wittman, B., Mathur, A. and Korb, T. (2013). Start Concurrent. Pup Department of Computer Science.